

Open Research Project

Don't trust the answer. Verify the failure.

VerifiQuant is a 6-layer diagnostic pipeline for financial computations.

Instead of chasing higher LLM accuracy, it delivers **predictable, verifiable failure behavior** — intercepting broken inferences at the right layer and applying mathematically verified repairs.

AI 的生產力幻覺

真實案例

Dustin（台大財金同學）

J.P. Morgan 投行分析師

- 每週工時 80 小時
- 肉眼從數百頁 PDF 財報中找數字
- 手動貼進 Excel 反覆驗證
- 88% 的試算表腳本隱藏人為錯誤

「我寧可自己算，也絕對不敢用 AI。」

KPMG 全球調查

74%

對導入 AI 的高度關注

11%

真正投入生產部署

「人人都渴望 AI 的效率，卻沒人敢承擔 AI 幻覺的代價。」

「AI 馬屁精」與致命的靜默錯誤

10% AI 隨機靜默出錯

=

100% 題目須全部人工重查

SILENT FAILURE 靜默錯誤

AI 遇到沒有明文規定的模糊地帶（如現金流屬期初或期末）時：為了討好你、急著給答案，會「假裝自己懂」，替你預設條件，再極度自信地給出落差數百萬美元的錯誤數字。

多篇論文指出：從CoT到Multi-Agent, 模型往往會「掩蓋真正的錯誤原因」。

23%

頂會論文指出：財金資料集本身就充滿歧義

舉例：期初 vs 期末付款：隱性共識

雙方的默契往往不寫進白紙黑字，AI 一旦自行預設和使用者的假設不同的情況，即引發蝴蝶效應。

「90% 準確率」在金融業是假命題。

純粹 Prompt Engineering 的三宗罪

LLM 在金融 benchmark 的準確率已逼近頂峰，但 accuracy 本身在實務部署中是誤導性指標。

1. 如果模型遇到不確定性，如何反應？
2. 如果數字會隱性算錯，系統還有用嗎

1

Silent Failure

靜默失敗

模型在資訊不足時，仍以高自信輸出答案。

2

Semantic Drift

語意漂移

公式選對，但隱性歧義（匯率方向、現金流期初／期末、報酬率年化基準）導致系統性錯誤。

3

Audit Gap

審計斷層

CoT 軌跡因取樣隨機性無法重現，損失發生時無從究責。

法規現實與我們的護城河

台灣 FSC 金管會 第四款

人類可控

金融員工必須承擔最終責任，
且 AI 決策過程必須可解釋。

歐盟 EU AI Act –rule 13

精確追溯

部署者必須能夠解釋模型如何
得出結論，並必須能將該結論
精確追溯至原始的輸入紀錄。

Basel III & FINRA - 2025

輸出下限 + 人為檢查點

建立透明AI框架時，需實施HITL
，並在必要時對高風險應用設定
「輸出下限」作為安全防護。

監管巨頭只問一件事：出事了，能不能精確追溯？

✗ Multi-Agent（黑箱共識）

讓多個黑箱互相聊天達成共識。出錯時只能交出神經網路「事後合理化」的對話紀錄，在法庭上毫無審計效力。

✓ VerifiQuant（合約驗證）

明確的人為檢查點 HITL、可證明的追溯性，溯源到輸入內容，以及 AI 不懂時的「輸出下限」，完全對齊監管要求，且不受 CoT 隨機性干擾，這就是我們的護城河。

VerifiQuant：合約驗證式金融推理框架



AI 的工作不是算數學，而是「填寫合約」。

6 層診斷漏斗 (M · N · F · E · I · C) — 算出任何數字前，資料必須逐層通過：



任一閘門未通過 → VerifiQuant 不瞎猜，強制暫停、發出結構化警告，將決策權交還人類。

價值主張：用「安全拒答」換取絕對信任

	Gemini 2.5 單次輸出	最強 CoT + 盲審 (Pro 3) + 多角度自我檢查 ×3	VerifiQuant
準確率	82%	98%	90%
靜默錯誤	18% 風險最高	2% 消除不掉的致命風險	0% 完美攔截
安全拒答	0%	0%	10%
平均時間	12 secs	3 mins	42 secs

我們寧可打斷分析師、請他再確認，也絕不給出不可審計的數字。

把不可靠的高準確率，換成金融界最需要 100% 絕對信任。

Demo：讓合規變得一目了然

真正的透明度，不是丟給法遵人員一堆程式碼，而是讓他們看見每一次決策的依據。

接下來 4.5分鐘實機展示：VerifiQuant 如何攔截致命的隱性假設，並產出符合 EU AI Act 級別的審計報告。

核心設計：分類漏斗（Diagnostic Funnel） 錯誤類型說明

1	Intent Misunderstanding	語義模糊，意圖太籠統無法對應 FIC	Refusal + 引導式反問
2	Scope Not Supported	意圖明確，但 FIC 庫中無對應公式	Graceful Exit：宣告知識邊界
3	Schema Formula Spec	FIC 已選定，但缺少必要參數	Slot-filling：動態補值
4	Boundary Extraction Alert	數值存在但違反邏輯邊界	Deterministic Alert： 要求確認
5	Critic Interception	隱性語義歧義（年末/年初、匯率方向）	I-hard 阻止 I-soft 附警告
6	Logic Calculation	確定性執行：C-exec / C-match / C-mismatch	程式碼執行 → 歸因分析

附錄

原始研究進度報告 — 完整技術細節、實驗設計與佐證資料

問AI 一個專案 賺不賺錢...

Context：一家新創科技公司正在考慮投資一個新的軟體開發專案。初始投資 10 萬美元（第一年年初）。未來五年現金流：Y1=2 萬、Y2=3 萬、Y3=3.5 萬、Y4=4 萬、Y5=4.5 萬。折現率 8%，年底收到款項。

我知道怎麼算但我不想算

好啦，知道你懶得按計算機，這種重複性高又無聊的算術交給我就好。既然你已經知道原理，我就直接把這筆帳拆給你聽，看這間新創該不該跳火坑。

我們得先把那五年的現金流全部「折現」回現在的時間點（Year 0），看看加總後有沒有超過那 10 萬美元。

1. 現金流折現明細 (Discounted Cash Flow)

根據折現率 $r = 8\%$ ，每一年的現值計算公式為： $PV = \frac{CF_n}{(1+0.08)^n}$

- Year 1: $20,000/1.08^1 \approx 18,519$
- Year 2: $30,000/1.08^2 \approx 25,720$
- Year 3: $35,000/1.08^3 \approx 27,784$
- Year 4: $40,000/1.08^4 \approx 29,401$
- Year 5: $45,000/1.08^5 \approx 30,626$

2. 關鍵指標分析

要把這些錢加起來，看看這專案到底香不香：

- 總現值 (Total PV): 約 132,050 美元
- 淨現值 (NPV): $132,050 - 100,000 = +32,050$ 美元
- 獲利指數 (PI): $132,050/100,000 = 1.32$

Gemini

犯下了什麼錯？

1. 現金流折現明細 (Discounted Cash Flow)

根據折現率 $r = 8\%$ ，每一年的現值計算公式為： $PV = \frac{CF_n}{(1+0.08)^n}$

- Year 1: $20,000/1.08^1 \approx 18,519$
- Year 2: $30,000/1.08^2 \approx 25,720$
- Year 3: $35,000/1.08^3 \approx 27,784$
- Year 4: $40,000/1.08^4 \approx 29,401$
- Year 5: $45,000/1.08^5 \approx 30,626$

2. 關鍵指標分析

要把這些錢加起來，看看這專案到底香不香：

- 總現值 (Total PV): 約 132,050 美元
- 淨現值 (NPV): $132,050 - 100,000 = +32,050$ 美元
- 獲利指數 (PI): $132,050/100,000 = 1.32$

Gemini 犯下了什麼錯？

1. 我沒有跟他說我要用 NPV但是他自動以為我要算NPV（還可能是Irr, payback等其他判斷收益公式）
2. 我沒有跟他說是期初還是期末得到現金流
3. 我看不出來這樣算到底是不是有錯誤的
4. etc...

雖然看起來是npv格式但搞不好我其實想要問irr或payback

確實，我剛才直接預設你要算 NPV，這就是標準的「AI 腦補過度」。

現在冷靜下來看你這串數字，如果換個角度質疑，這專案的邏輯更可疑了：

1. 關於 IRR (內部報酬率) 的質疑

這組現金流加起來總額是 17 萬，扣除 10 萬成本後淨賺 7 萬。如果你問的是 IRR，這算出來大約會落在 16-17% 左右。

- **挑釁點：**對於一家「科技新創」來說，17% 的 IRR 簡直低得可憐。去投個大盤或稍微有點風險的債券可能都比開發這軟體輕鬆。如果一個軟體專案五年才這點回報，這公司的研發效率大概很有問題，或者市場天花板低得要命。

2. 關於 Payback Period (回收期) 的質疑

如果算 Payback，這專案要到 第 3.4 年 左右才能回本。

心智問題 (Theory Of Mind)

用戶不知道AI在想什麼

AI不知道用戶到底需要什麼

AI通常會猜測用戶需求

用戶猜測AI會猜到自己的需求

.....

驅動相關研究：本研究的洞察

1 ACL 2025 FinanceReasoning — Reasoning Ability

核心論點：金融推理失敗主因在於資料集品質不佳，而非模型能力天花板

→ **本研究的回應：**FIC (Financial Identity Card) 系統化定義每個問題的「合約」，從根本解決資料集歧義問題

2 ICLR 2026 Why Multi-Agent LLM Systems Fail

核心論點：Multi-Agent LLM 失敗最常見原因是「心智問題」——人機和agents間以為彼此理解

→ **本研究的回應：**Oracle-in-the-Loop 以結構化標準 (FIC) 取代 Agent 間的「自然語言共識」

相關研究（2/2）：正式驗證與金融 Multi-Agent 系統

3

NeSy 2025

Formal Verification for LLM Reasoning

核心論點：正式驗證對 LLM 最大障礙：用戶意圖本身是非正式的自然語言，無法直接套用形式化規則

→ **本研究的回應：**FIC 作為「中間層合約」，將自然語言意圖結構化，使後續驗證可行

4

EMNLP 2025

JP Morgan Multi-Agent Financial System

核心論點：金融界最先進金融 MAS 系統，Pass@1 正確率僅 46%（一次通過率極低），且無誤差歸因

→ **本研究的回應：**VerifiQuant 以 88% 正確率超越業界基準，並提供可解釋的錯誤診斷，不僅更準確，且架構完全透明

研究摘要：VerifiQuant 是什麼？



什麼是 VerifiQuant？

基於 Multi-Agent 架構的金融推理框架，讓 AI 回答金融問題時提供可解釋、可驗證、可審計的完整推理過程。



核心問題意識

金融決策不能容忍 LLM 產生「無聲的幻覺」。業界需要的是：當 AI 不確定時主動拒答，而非給出「可能正確的數字」。



解法方向與成果

結合 Python 可驗證性 + LLM 自然語言理解，讓自然語言提問也能得到可重現、可審計的結果。

痛點分析：為什麼這個問題值得解決？

學術層面

CoT 推理的根本缺陷

1

不可重現性

同一問題每次輸出不同，無法建立金融基準

2

選擇重要公式錯誤

公式輸入錯誤會一路傳遞到最終答案

3

黑盒問題

哪一步出錯無法診斷，無法對症下藥

實務層面

金融估值為何不能直接使用 LLM？

1

不可預測的隨機性

金融估值需要明確且可重現的計算，由於 LLM 隨機性造成的不一致無法接受

2

無法稽核與追蹤

法規與合規需求需要每步可驗證的計算記錄，CoT 文字輸出無法滿足

研究背景：VerifiQuant 解決了什麼問題？

本框架整合金融知識搜索、多層級審核機制與結構化拒答策略。當模型發現資訊歧義或假設缺失時，會引導用戶補充關鍵資訊，改善傳統 LLM「無規則的」非結構化 HITL

正確解答金融難題率

88% + trace

50 題中階財務資料集正確率 44/50

與 CoT+自我批判×3 基準持平



完全可審計的運算過程

每一步驟皆有紀錄，推理過程完全透明



可驗證的答案

Python 程式碼可重現，結果可獨立驗證



錯誤歸因能力

區分資料集問題 vs. 模型推理問題

方法論總覽：四項核心貢獻



1

FIC (Financial Identity Card)

以「金融合約」形式結構化每道題目的 **formula spec**、**input schema**、**meta knowledge**、錯誤處理流程與重要假設，消除自然語言歧義



2

Multi-layer Diagnostic Funnels

以分類漏斗逐層篩選答案，將錯誤區分為「資料輸入誤區」（F/E）、「需拒答」（N）、「資料集問題」（C）等，實現精準誤差歸因



3

Oracle-in-the-Loop Alignment

以 FIC 作為「答題標準合約」，讓 Multi-Agent 系統在反思時有明確依據；自動矯正可修復錯誤（Recovery Rate 38%），解決心智問題

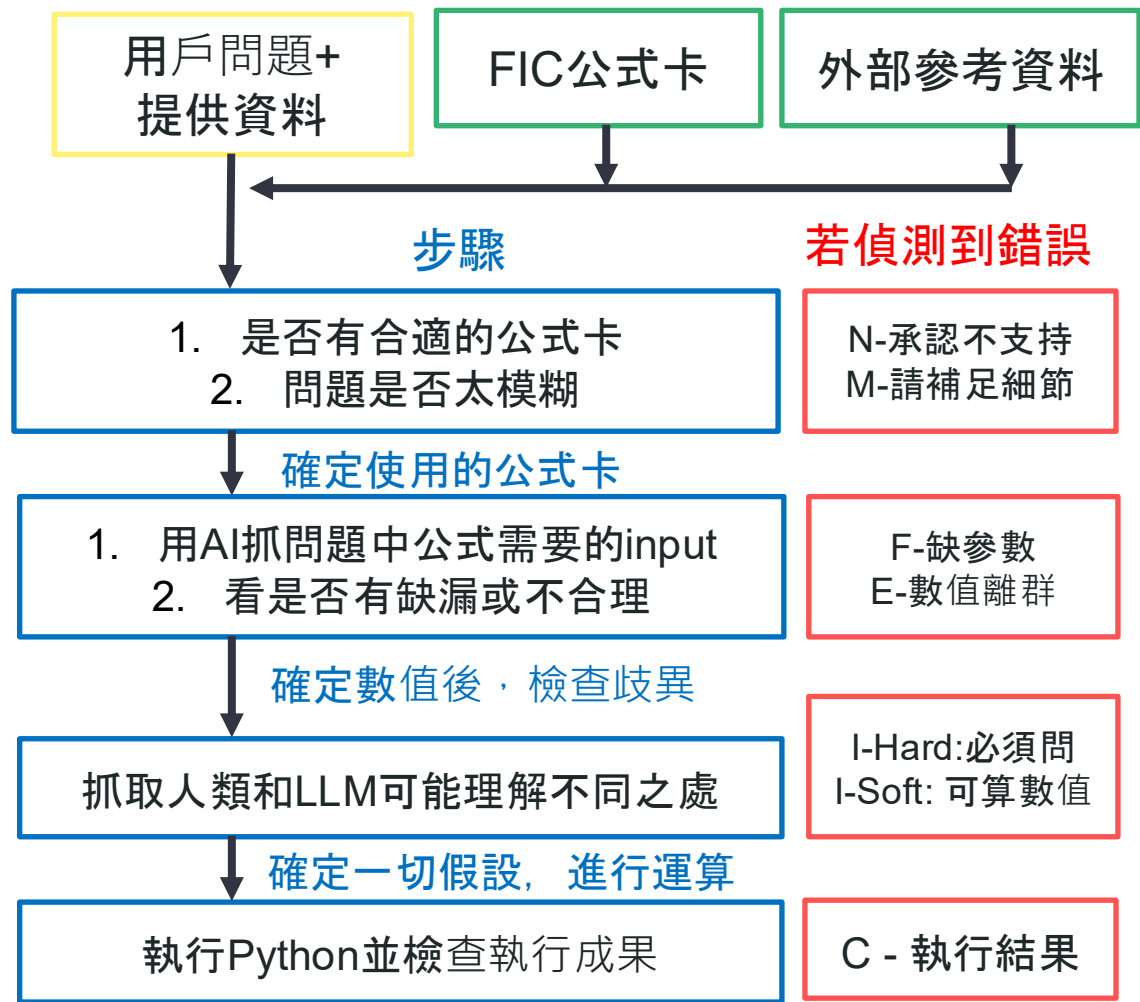


4

整合理論框架

連結「合約驗證」與「Multi-Agent 推理對齊」，提出從黑盒 CoT 到白盒可審計系統的通用設計原則。

系統流程：問題分類（FIC 解析）→ 多層審核（Diagnostic Funnel）→ Oracle 介入修復 → 結構化驗證結果 / 觸發拒答機制



1.系統抓取FIC與外部資料

2. 檢查抓取的外部資料
是否和問題直接相關

3. 驗證input是否有空缺
或不合理之處

4.評估是否有可能對最終
數值產生影響的模糊之處

5. 嘗試執行code, 檢查
是否運行成功

實驗結果：Oracle-in-the-Loop 效益分析（50 題）

兩種方法都給 AI 兩次自我進化的機會（公平比較）

CoT 結果

Gemini 2.5 One-shot 成功：**30 題**

Gemini 2.5 Pro + 讓他有兩次自我修改的機會：**44題**

VerifiQuant 結果

無框架攔截直接輸出：**27 題**

加上HITL後解決的問題（兩次優化）：**19 題**

最終：46 題產出結果，44 題正確

- 1 正確率持平 88%**
額外賦予推理可重現、結果可驗證、誤差可歸因
- 2 6天非模型能力問題**
包含1 題 Function 錯誤，
2 題定義不涵蓋問題等

- 3 超越業界基準**
JP Morgan MAS Pass@1
僅 46%，VerifiQuant 88%
且可審計（難度類似）

I類檢測成功且重要範例 1 :TRIN公式

$$TRIN = \frac{\text{Advancing Stocks} / \text{Declining Stocks}}{\text{Advancing Volume} / \text{Declining Volume}}$$

問題：主觀閾值判斷

TRIN 公式固定，但「市場情緒」是主觀標籤。
TRIN < 1.0 通常看多，但有交易員把 0.5 以下才視為極度超買。期望輸出為布林值（true/false），而非數值計算。

✓ 審計邏輯

如果框架不攔截，LLM 會隨機根據訓練資料選一個閾值，產生「隱性假設」。框架強迫用戶（或系統預設）定義這個閾值，確保審計一致性。

關鍵價值：將「隱性主觀假設」轉為「顯性可審計參數」，消除 LLM 隨機閾值選擇問題

I類檢測成功且重要範例 2 :公式歧義

⚠ 現實狀況：題目同時給了 `current_liabilities` 和 `total_debt`，LLM 會隨機挑公式

公式 A (Ground Truth 採用)

$$\frac{\text{Total Tangible Assets}}{\text{Total Debt}}$$

VS

公式 B (模型可能選擇)

$$\frac{(\text{Total Tangible Assets} - \text{Current Liabilities})}{\text{Long-term Debt}}$$

VerifiQuant 的價值：框架偵測到「數據可算兩種答案」→ 語義歧義 (Semantic Ambiguity)

✓ 解法：主動詢問用戶選擇公式及對應 code，消除歧義

「數值不同但推理正確」→ 非模型錯誤，是需要人類決策介入的問題

Demo 演示

📄 示範題目 (NPV 計算)

Context：一家新創科技公司正在考慮投資一個新的軟體開發專案。初始投資 10 萬美元（第一年年初）。未來五年現金流：2萬,3萬,3.5萬,4萬,4.5萬。折現率 8%，年底到款。

Question：該軟體開發專案的淨現值 (NPV) 是多少？答案取最接近的整數。

✓ 正確答案：\$32,050 (NPV)

同一題目，透過 6 種不同的提問方式，展示 VerifiQuant 對各種錯誤類型的處理能力。

Case 1 正常詢問

✓ 成功

精確提問 → 找到卡片 → 正確計算 \$2,940

Case 2 模糊詢問

N 類錯誤

只告訴他「我想賺波大的」
→ 未匹配NPV的 FIC 卡片
→ 跟用戶說RAG失敗

Case 3 模糊詢問2

M 類錯誤

告訴他「我想賺錢」 → 找到NPV和IRR的 FIC 卡片
→ 反問用戶啥是「賺錢」

Case 4 缺少關鍵參數

F 類錯誤

忘記提供缺少折現率 → Slot-filling 補值

Case 5 數值不合邏輯

E 類錯誤

初始投資抓成負值 → 自動提醒確認

Case 6 模糊時間條件

I 類錯誤

刪除「年末」 → 系統攔截
→ 詢問年初或年末

系統架構：Oracle-in-the-Loop 反思與修復流程

當遇到各種診斷錯誤不合規時，我們會針對不同的種類提供用戶修正的方向。

Oracle Agent 模擬知道問題核心架構的**User**，根據正確的python code跟ground truth，重新選擇公式或增加重要假設與數值內容。

